

Rapport Complet de Fin de TP : Microservices, API Gateway & Docker

Solution Globale Clé en Main — Conforme aux exigences du Module Cloud Native

Partie 1 : Structure Globale de l'Arborescence

Voici la structure finale exacte attendue à la racine de votre projet global TP_5 :

```
TP_5/
├── docker-compose.yml
├── gateway/
│   ├── Dockerfile
│   ├── index.js
│   └── package.json
├── livre-service/
│   ├── Dockerfile
│   ├── index.js
│   └── package.json
├── membre-service/
│   ├── Dockerfile
│   ├── index.js
│   └── package.json
└── emprunt-service/
    ├── Dockerfile
    ├── index.js
    └── package.json
```

Partie 2 : Fichier d'Orchestration Global (Racine)

 **Fichier : TP_5/docker-compose.yml**

```
version: '3.8'

services:
  mongodb:
    image: mongo:latest
    container_name: mongodb_container
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
    networks:
      - AppNetwork

  livre-service:
    build: ./livre-service
    container_name: livre_service_container
    ports:
      - "3001:3001"
    environment:
      - MONGO_URL=mongodb://mongodb:27017/bibliotheque_livres
    depends_on:
```

```

    - mongodb
networks:
    - AppNetwork

membre-service:
    build: ./membre-service
    container_name: membre_service_container
    ports:
        - "3002:3002"
    environment:
        - MONGO_URL=mongodb://mongodb:27017/bibliotheque_membres
    depends_on:
        - mongodb
    networks:
        - AppNetwork

emprunt-service:
    build: ./emprunt-service
    container_name: emprunt_service_container
    ports:
        - "3003:3003"
    environment:
        - MONGO_URL=mongodb://mongodb:27017/bibliotheque_emprunts
        - LIVRE_SERVICE_URL=http://livre-service:3001
        - MEMBRE_SERVICE_URL=http://membre-service:3002
    depends_on:
        - mongodb
        - livre-service
        - membre-service
    networks:
        - AppNetwork

gateway:
    build: ./gateway
    container_name: api_gateway_container
    ports:
        - "3000:3000"
    environment:
        - LIVRE_SERVICE_URL=http://livre-service:3001
        - MEMBRE_SERVICE_URL=http://membre-service:3002
        - EMPRUNT_SERVICE_URL=http://emprunt-service:3003
    depends_on:
        - livre-service
        - membre-service
        - emprunt-service
    networks:
        - AppNetwork

volumes:
    mongo_data:

networks:
    AppNetwork:
        driver: bridge

```

Partie 3 : Recette de Construction Unique (Le Dockerfile)

Créez exactement le même fichier nommé Dockerfile dans **chacun** des 4 dossiers de vos services.

 **Fichier : [service-name]/Dockerfile**

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --only=production
COPY . .
CMD ["node", "index.js"]
```

Partie 4 : Code Source Intégral Adapté pour Docker

1. Service Livre

 **Fichier : livre-service/index.js (Port 3001)**

```
const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();
app.use(express.json());

const url = process.env.MONGO_URL || "mongodb://mongodb:27017/bibliotheque_livres";
const client = new MongoClient(url);
let db;

client.connect().then(() => {
  db = client.db();
  console.log("MongoDB connectée pour Livre Service");
}).catch(err => console.log(err));

app.get('/livres', async (req, res) => {
  try {
    const livres = await db.collection('livres').find({}).toArray();
    res.status(200).json(livres);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

app.get('/livres/:id', async (req, res) => {
  try {
    const livre = await db.collection('livres').findOne({ id: parseInt(req.params.id) });
    if (!livre) return res.status(404).json({ message: "Livre non trouvé" });
    res.status(200).json(livre);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

app.post('/livres', async (req, res) => {
```

```

    try {
      const counter = await db.collection('livres').find({}).toArray();
      const nextId = counter.length > 0 ? Math.max(...counter.map(l => l.id || 0)) + 1 : 1;
      const newLivre = { id: nextId, ...req.body, disponible: true };
      await db.collection('livres').insertOne(newLivre);
      res.status(201).json(newLivre);
    } catch (err) {
      res.status(500).json({ message: err.message });
    }
  });

  app.put('/livres/:id', async (req, res) => {
    try {
      await db.collection('livres').updateOne({ id: parseInt(req.params.id) }, { $set:
req.body });
      res.status(200).json({ message: "Livre mis à jour" });
    } catch (err) {
      res.status(500).json({ message: err.message });
    }
  });

  app.patch('/livres/:id/disponibilite', async (req, res) => {
    try {
      const { disponible } = req.body;
      await db.collection('livres').updateOne({ id: parseInt(req.params.id) }, { $set:
{ disponible } });
      res.status(200).json({ message: "Disponibilité modifiée" });
    } catch (err) {
      res.status(500).json({ message: err.message });
    }
  });

  app.delete('/livres/:id', async (req, res) => {
    try {
      await db.collection('livres').deleteOne({ id: parseInt(req.params.id) });
      res.status(200).json({ message: "Livre supprimé" });
    } catch (err) {
      res.status(500).json({ message: err.message });
    }
  });

  app.listen(3001, () => console.log('Livre service actif sur le port 3001'));

```

2. Service Membre

 **Fichier : membre-service/index.js (Port 3002)**

```
const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();
app.use(express.json());

const url = process.env.MONGO_URL || "mongodb://mongodb:27017/bibliotheque_membres";
const client = new MongoClient(url);
let db;

client.connect().then(() => {
  db = client.db();
  console.log("MongoDB connectée pour Membre Service");
}).catch(err => console.log(err));

app.get('/membres/:id', async (req, res) => {
  try {
    const membre = await db.collection('membres').findOne({ id: parseInt(req.params.id) });
    if (!membre) return res.status(404).json({ message: "Membre non trouvé" });
    res.status(200).json(membre);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

app.post('/membres', async (req, res) => {
  try {
    const counter = await db.collection('membres').find({}).toArray();
    const nextId = counter.length > 0 ? Math.max(...counter.map(m => m.id || 0)) + 1 : 1;
    const newMembre = { id: nextId, ...req.body };
    await db.collection('membres').insertOne(newMembre);
    res.status(201).json(newMembre);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

app.listen(3002, () => console.log('Membre service actif sur le port 3002'));
```

3. Service Emprunt (Orchestrateur HTTP)

 **Fichier : emprunt-service/index.js (Port 3003)**

```
const express = require("express");
const { MongoClient } = require("mongodb");
const axios = require("axios");

const app = express();
app.use(express.json());

const url = process.env.MONGO_URL || "mongodb://mongodb:27017/bibliotheque_emprunts";
const LIVRE_SERVICE = process.env.LIVRE_SERVICE_URL || 'http://livre-service:3001';
```

```

const MEMBRE_SERVICE = process.env.MEMBRE_SERVICE_URL || 'http://membre-service:3002';

const client = new MongoClient(url);
let db;

client.connect().then(() => {
  db = client.db();
  console.log("MongoDB connectée pour Emprunt Service");
}).catch(err => console.log(err));

app.post('/emprunts', async (req, res) => {
  try {
    const { idLivre, idMembre } = req.body;

    // Validation Inter-service via Axios (Utilisation des DNS Docker)
    const livreRes = await axios.get(`${LIVRE_SERVICE}/livres/${idLivre}`);
    if (!livreRes.data.disponible) {
      return res.status(400).json({ message: "Le livre demandé n'est pas disponible" });
    }

    await axios.get(`${MEMBRE_SERVICE}/membres/${idMembre}`);

    const counter = await db.collection('emprunts').find({}).toArray();
    const nextId = counter.length > 0 ? Math.max(...counter.map(e => e.id || 0)) + 1 : 1;

    const newEmprunt = { id: nextId, idLivre, idMembre, retourne: false, dateEmprunt: new
Date() };
    await db.collection('emprunts').insertOne(newEmprunt);

    // Mettre à jour la disponibilité du livre
    await axios.patch(`${LIVRE_SERVICE}/livres/${idLivre}/disponibilite`, { disponible:
false });

    res.status(201).json(newEmprunt);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

app.patch('/emprunts/:id/retour', async (req, res) => {
  try {
    const id = parseInt(req.params.id);
    const { retourne } = req.body;

    const emprunt = await db.collection('emprunts').findOne({ id: id });
    if (!emprunt) return res.status(404).json({ message: "Emprunt non trouvé" });

    await db.collection('emprunts').updateOne({ id: id }, { $set: { retourne } });
    await axios.patch(`${LIVRE_SERVICE}/livres/${emprunt.idLivre}/disponibilite`,
{ disponible: true });

    res.status(200).json({ message: "Livre retourné avec succès" });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

app.listen(3003, () => console.log('Emprunt service actif sur le port 3003'));

```

4. API Gateway

 **Fichier : gateway/index.js (Port Unique Public 3000 — Sans Sécurité)**

```
const express = require("express");
const axios = require("axios");

const app = express();
app.use(express.json());

const LIVRE_SERVICE = process.env.LIVRE_SERVICE_URL || 'http://livre-service:3001';
const MEMBRE_SERVICE = process.env.MEMBRE_SERVICE_URL || 'http://membre-service:3002';
const EMPRUNT_SERVICE = process.env.EMPRUNT_SERVICE_URL || 'http://emprunt-service:3003';

// ROUTAGE LIVRES
app.get('/livres', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres`);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.get('/livres/:id', async (req, res) => {
  try {
    const r = await axios.get(`${LIVRE_SERVICE}/livres/${req.params.id}`);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.post('/livres', async (req, res) => {
  try {
    const r = await axios.post(`${LIVRE_SERVICE}/livres`, req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.put('/livres/:id', async (req, res) => {
  try {
    const r = await axios.put(`${LIVRE_SERVICE}/livres/${req.params.id}`, req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.patch('/livres/:id/disponibilite', async (req, res) => {
  try {
    const r = await axios.patch(`${LIVRE_SERVICE}/livres/${req.params.id}/disponibilite`,
```

```

req.body));
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.delete('/livres/:id', async (req, res) => {
  try {
    const r = await axios.delete(`${LIVRE_SERVICE}/livres/${req.params.id}`);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

// ROUTAGE MEMBRES
app.get('/membres/:id', async (req, res) => {
  try {
    const r = await axios.get(`${MEMBRE_SERVICE}/membres/${req.params.id}`);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.post('/membres', async (req, res) => {
  try {
    const r = await axios.post(`${MEMBRE_SERVICE}/membres`, req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

// ROUTAGE EMPRUNTS
app.post('/emprunts', async (req, res) => {
  try {
    const r = await axios.post(`${EMPRUNT_SERVICE}/emprunts`, req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

app.patch('/emprunts/:id/retour', async (req, res) => {
  try {
    const r = await axios.patch(`${EMPRUNT_SERVICE}/emprunts/${req.params.id}/retour`,
req.body);
    res.status(r.status).json(r.data);
  } catch (err) {
    if (err.response) return res.status(err.response.status).json(err.response.data);
    res.status(500).json({ message: err.message });
  }
});

```



```
app.listen(3000, () => console.log('Gateway active sur le port 3000'));
```

Partie 5 : Commandes d'Exécution Consolides (Terminal)

Ouvrez un terminal unique pointant sur le dossier principal TP_5 :

1. **Lancement initial de compilation globale :**
`docker-compose build`
2. **Déploiement complet en arrière-plan :**
`docker-compose up -d`
3. **Vérification instantanée de l'état du cluster :**
`docker-compose ps`
4. **Affichage des logs d'un service (ex: gateway) pour débbugger :**
`docker-compose logs -f gateway`
5. **Arrêt propre de toute l'infrastructure :**
`docker-compose down`